

Programación Lógica

Estructuras incompletas



Instituto de Computación - Facultad de Ingeniería

Estructuras incompletas

- Técnica de representación de datos y programación que implica simplificaciones y aumento de eficiencia.
- Varios tipos y aplicaciones
 - Listas de diferencias
 - Colas
 - Listas incompletas
 - Árboles binarios incompletos

Estructuras incompletas

Listas de diferencias

- Motivación: ¿qué pasa si queremos insertar al final de una lista?

Estructuras incompletas

Listas de diferencias

- Motivación: ¿qué pasa si queremos insertar al final de una lista?

- Ineficiencia:

```
insertar([], E, [E]).
```

```
insertar([X|Xs], E, [X|XsConE]) :-
```

```
    insertar(Xs, E, XsConE).
```

- El problema se generaliza para append.
 - Podría “apuntar” al final de la lista.

Estructuras incompletas

Listas de diferencias

- Cualquier lista puede ser vista como la diferencia de 2 listas (entendemos la diferencia de listas como una inversa de la concatenación):

$$[a, b, c] = [a, b, c, d, e] - [d, e]$$

$$[a, b, c] = [a, b, c | [8]] - [8]$$

$$[a, b, c] = [a, b, c] - []$$

Estructuras incompletas

Listas de diferencias

- Cualquier lista puede ser vista como la diferencia de 2 listas (entendemos la diferencia de listas como una inversa de la concatenación):

$$[a, b, c] = [a, b, c, d, e] - [d, e]$$

$$[a, b, c] = [a, b, c | [8]] - [8]$$

$$[a, b, c] = [a, b, c] - []$$

- Todos los casos anteriores son instancias de:

$$[a, b, c | X] - X$$

Estructuras incompletas

Listas de diferencias

- Cualquier lista puede ser vista como la diferencia de 2 listas (entendemos la diferencia de listas como una inversa de la concatenación):

$$[a, b, c] = [a, b, c, d, e] - [d, e]$$

$$[a, b, c] = [a, b, c | [8]] - [8]$$

$$[a, b, c] = [a, b, c] - []$$

- Todos los casos anteriores son instancias de:

$$[a, b, c | X] - X$$

- O sea que cualquier lista es la diferencia de dos listas incompletas.

Estructuras incompletas

Listas de diferencias

- Representamos una lista de diferencias por un término estructurado o por 2 términos.
- El punto clave es tener acceso al resto variable de la lista, para eso se repite la misma variable.
- Algunas notaciones usuales son:

$X - Rx$

$X \setminus Rx$

$dl(X, Rx)$

X, Rx

siendo $X = [x1, x2, \dots, xn \mid Rx]$

Estructuras incompletas

Listas de diferencias

Observaciones:

- La lista $[a, b, c | X] - X$ representa lo mismo que la lista $[a, b, c]$ pero no son términos equivalentes.
- El guión es solo notación, no implica la aplicación de una operación de resta sobre las listas.
- Por lo tanto $[a, b, c | X] - X$ y $[a, b, c]$ **NO UNIFICAN.**
- La lista vacía se representa como $X - X$.

Estructuras incompletas

Listas de diferencias

- Es posible concatenar 2 listas de diferencias en tiempo cte. (1 paso de resolución).

`append_dl(A - Ra, B - Rb, A - Rb) : - B = Ra.`

Estructuras incompletas

Listas de diferencias

- Es posible concatenar 2 listas de diferencias en tiempo cte. (1 paso de resolución).

`append_dl(A - Ra, B - Rb, A - Rb) : - B = Ra .`

- O directamente:

`append_dl(A - B, B - R, A - R) .`

Estructuras incompletas

Listas de diferencias

- Es posible concatenar 2 listas de diferencias en tiempo cte. (1 paso de resolución).

`append_dl(A - Ra, B - Rb, A - Rb) : - B = Ra .`

- O directamente:

`append_dl(A - B, B - R, A - R) .`

- B debe ser unificable con Ra (listas compatibles).
- Siempre son compatibles si Ra es una variable.

Estructuras incompletas

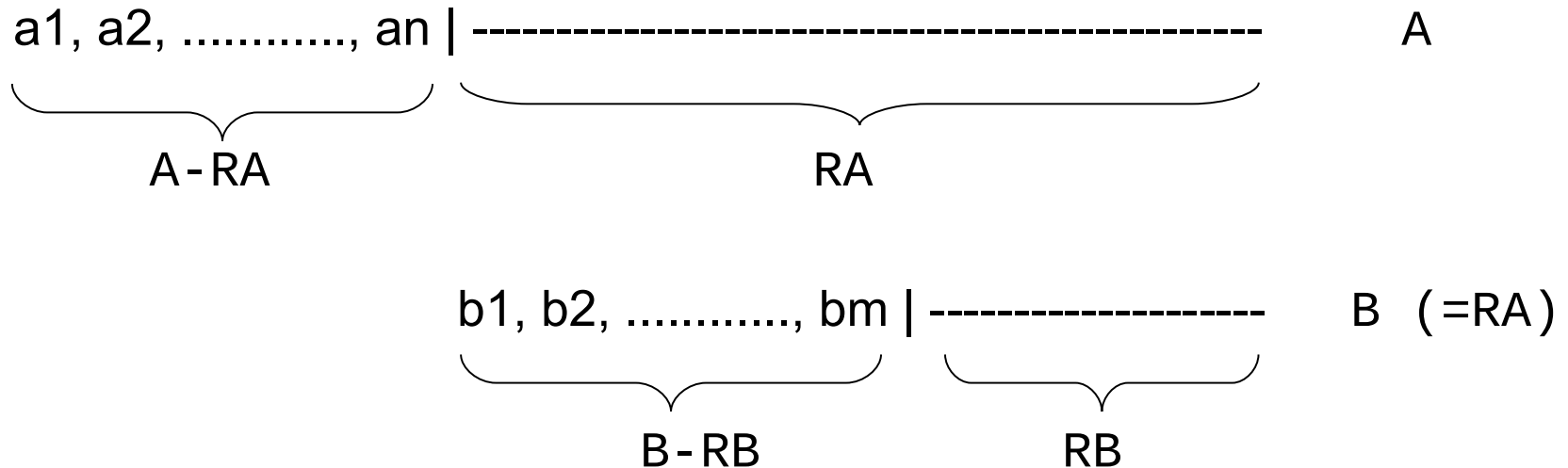
Listas de diferencias

$a_1, a_2, \dots, a_n \mid \text{-----} \quad A$

$\underbrace{\hspace{10em}}_{A-RA} \quad \underbrace{\hspace{10em}}_{RA}$

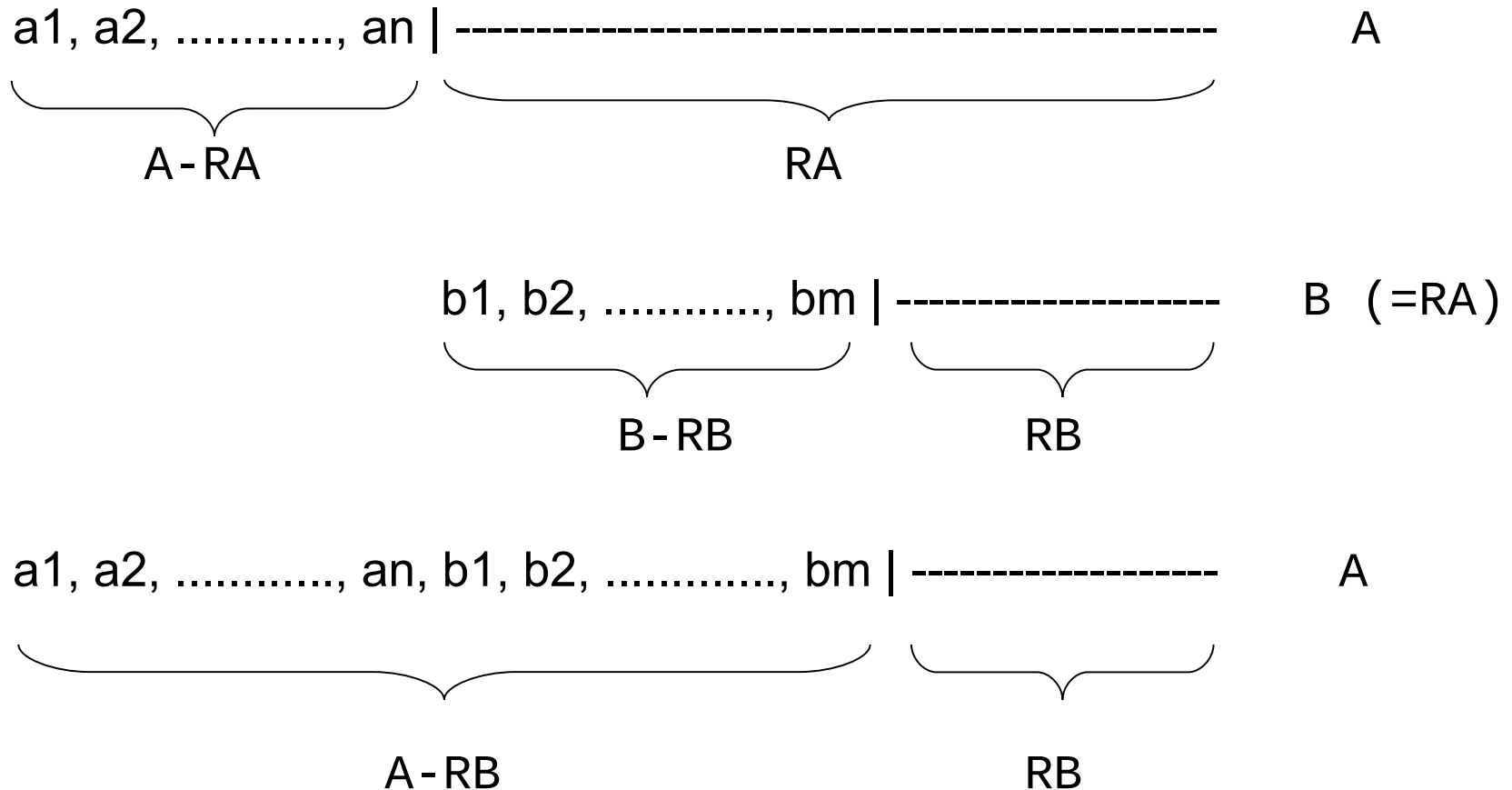
Estructuras incompletas

Listas de diferencias



Estructuras incompletas

Listas de diferencias



Estructuras incompletas

Listas de diferencias

- Para poder usar `append_dl` los argumentos **tienen** que ser listas de diferencias.
- No es inmediato pasar de una lista “normal” a una de diferencias:

```
%lista_dl(Lista,Lista_dl) <- Lista_dl es la versión  
                             lista de diferencias de  
                             Lista
```

Estructuras incompletas

Listas de diferencias

- Para poder usar `append_dl` los argumentos **tienen** que ser listas de diferencias.
- No es inmediato pasar de una lista “normal” a una de diferencias:

```
%lista_dl(Lista,Lista_dl) <- Lista_dl es la versión  
                           lista de diferencias de  
                           Lista
```

```
lista_dl([], Rx-Rx).  
lista_dl([E|Es], [E|X]-Rx) :-  
    lista_dl(Es, X-Rx).
```

Estructuras incompletas

Listas de diferencias

- Para poder usar `append_dl` los argumentos **tienen** que ser listas de diferencias.
- No es inmediato pasar de una lista “normal” a una de diferencias:

```
%lista_dl(Lista,Lista_dl) <- Lista_dl es la versión  
                             lista de diferencias de  
                             Lista
```

```
lista_dl([], Rx-Rx).  
lista_dl([E|Es], [E|X]-Rx) :-  
    lista_dl(Es, X-Rx).
```

Tengo que recorrer
la lista!

Estructuras incompletas

Listas de diferencias - ejemplos

- Insertar un elemento al final
 $\text{ins_final}(\text{LA}, \text{X}, \text{LA2}) \leftarrow \text{LA2}$ es la lista de diferencias LA incluyendo el elemento X al final
- Insertar un elemento al principio
 $\text{ins_inicio}(\text{LA}, \text{X}, \text{LA2}) \leftarrow \text{LA2}$ es la lista de diferencias LA incluyendo el elemento X al principio
- Pasar de lista de diferencias a lista normal
 $\text{lista_comun}(\text{LA}, \text{L}) \leftarrow \text{L}$ es una lista común con los mismos elementos que la lista de diferencias LA

Estructuras incompletas

Listas de diferencias

- Pasar una lista normal a una lista de diferencias es costoso.
- Pero no siempre es imprescindible hacer el pasaje para aprovechar las ventajas de las listas de diferencias.
- Veremos algunos ejemplos de programas que utilizan *append*, para los cuales no es necesario convertir la lista primero.

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización

```
reverse(L, LRev) :-  
    reverse_dl(L, LRev-[]).
```

```
quicksort(L, LOrd) :-  
    quicksort_dl(L, LOrd-[]).
```

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización

```
reverse(L, LRev) :-    % naive_reverse usa append  
    reverse_dl(L, LRev-[]).
```

```
quicksort(L, LOrd) :- % se ordena la lista por partes  
    quicksort_dl(L, LOrd-[]). % y luego se usaría  
                                % append
```

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización: Reverse

```
reverse(L, LRev) :-  
    reverse_dl(L, LRev - []).  
  
reverse_dl([], Rx - Rx).  
reverse_dl([X|Xs], L - Rx) :-  
    reverse_dl(Xs, L - [X|Rx]).
```

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización: Reverse

```
reverse(L, LRev) :-  
    reverse_dl(L, LRev - []).
```

```
reverse_dl([], Rx - Rx).    % similitud con acumuladores  
reverse_dl([X|Xs], L - Rx) :-  
    reverse_dl(Xs, L - [X|Rx]).
```

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización: Reverse

```
reverse(L, LRev) :-                               % si L es [a,b,c]
    reverse_dl(L, LRev-[]). % LRev queda [c,b,a|[ ]] que
                           % equivale a [c,b,a]

reverse_dl([], Rx-Rx).
reverse_dl([X|Xs], L-Rx) :-
    reverse_dl(Xs, L-[X|Rx]).
```

Estructuras incompletas

?- reverse_dl([a,b,c], Y-Ry).

\

{X1/a, Xs1/[b,c], L1/Y, Rx1/Ry}

?- reverse_dl([b,c], Y - [a | Ry])

\

{X2/b, Xs2/[c], L2/Y, Rx2/[a | Ry]}

?- reverse_dl([c], Y - [b | [a | Ry]])

\

{X3/c, Xs3/[], L3/Y, Rx3/[b,a | Ry]}

?- reverse_dl([], Y - [c | [b,a | Ry]])

/

Y = [c,b,a | Ry]

```
reverse_dl([], Rx-Rx).
```

```
reverse_dl([X|Xs], L-Rx):-
```

```
    reverse_dl(Xs, L-[X|Rx]).
```

Estructuras incompletas

Listas de diferencias

Ejemplos de utilización: Quicksort

```
quicksort(L, LOrd) :-  
    quicksort_dl(L, LOrd-[]).
```

```
quicksort_dl([], Rx-Rx).  
quicksort_dl([X|Xs], Y-Rz) :-  
    particion(Xs, X, Menores, Mayores),  
    quicksort_dl(Menores, Y-[X|Z]),  
    quicksort_dl(Mayores, Z-Rz).
```

No necesito append!

Estructuras incompletas

Listas de diferencias

¿Cómo definimos el predicado **member** para una lista de diferencias?

```
%member_dl(E, X-Rx) <- E pertenece a la lista de  
                        diferencias X-Rx
```

Estructuras incompletas

Listas de diferencias

¿Cómo definimos el predicado **member** para una lista de diferencias?

```
%member_dl(E, X-Rx) <- E pertenece a la lista de  
                        diferencias X-Rx
```

Posible solución:

```
member_dl(E, [E|_] - _).  
member_dl(E, [E1|X] - Rx) :-  
    E \= E1,  
    member_dl(E, X - Rx).
```

Estructuras incompletas

Listas de diferencias

¿Cómo definimos el predicado **member** para una lista de diferencias?

```
%member_dl(E, X-Rx) <- E pertenece a la lista de  
                        diferencias X-Rx
```

Posible solución:

```
member_dl(E, [E|_] - _).  
member_dl(E, [E1|X] - Rx) :-  
    E \= E1,  
    member_dl(E, X-Rx).
```

PROBLEMA:

Si E **no** está en la lista, al llegar la recursión al paso base, el resto variable queda unificado con [E|_]. Entonces se agrega E a la lista y el predicado NO FALLA.

Estructuras incompletas

Listas de diferencias

¿Cómo definimos el predicado **member** para una lista de diferencias?

```
%member_dl(E, X-Rx) <- E pertenece a la lista de  
                        diferencias X-Rx
```

Una solución:

```
member_dl(E, [E|X] - _) :-  
    \+ var(X).  
member_dl(E, [E|Rx] - Rx) :-  
    var(Rx).  
member_dl(E, [E1|X] - Rx) :-  
    E \= E1,  
    member_dl(E, X-Rx).
```

Estructuras incompletas

Listas de diferencias para representar Colas

- Inicializamos la cola:

`vacía(C-C) .`

- Los elementos se insertan al final:

`insertar(E, C-[E|Y], C-Y) .`

- Los elementos siempre se sacan del principio:

`sacar(E, [E|X]-Y, X-Y) .`

Estructuras incompletas

Listas incompletas

Lista con un resto variable.

Ej.: $[a, b, c, d | X]$

Predicado para insertar un elemento E al final de la lista, sin repetirlo:

```
insSinRepetir(X, [X|Xs]).  
insSinRepetir(X, [X1|Xs]) :-  
    X \= X1,  
    insSinRepetir(X, Xs).
```

Estructuras incompletas

Listas incompletas

Ej.: $[a, b, c, d | X]$

- Predicado para insertar un elemento E al final de la lista, sin repetirlo:

```
insSinRepetir(X, [X|Xs]).  
insSinRepetir(X, [X1|Xs]) :-  
    X \= X1,  
    insSinRepetir(X, Xs).
```
- El elemento queda dentro de la misma lista que usamos como parámetro de entrada, no necesitamos otra lista como parámetro de salida.
- Usamos el “hueco” que dejamos al final de la lista incompleta para agregar allí el elemento.

Estructuras incompletas

Árboles incompletos

- En lugar de usar una constante para representar el árbol vacío (ver ej. 6 del pr. 1), uso una variable.
- Ej.: `abb(3, abb(1, A, B), abb(6, C, abb(7, D, E)))`
- Al igual que con la lista incompleta, si queremos agregar un elemento, lo insertamos en alguno de los “huecos” representados por las variables A, B, C, D y E.
- No tenemos que retornar otro árbol con el nuevo elemento.

Estructuras incompletas

Árboles incompletos

Ejercicio:

Escribir un predicado que agregue un elemento en un abb.